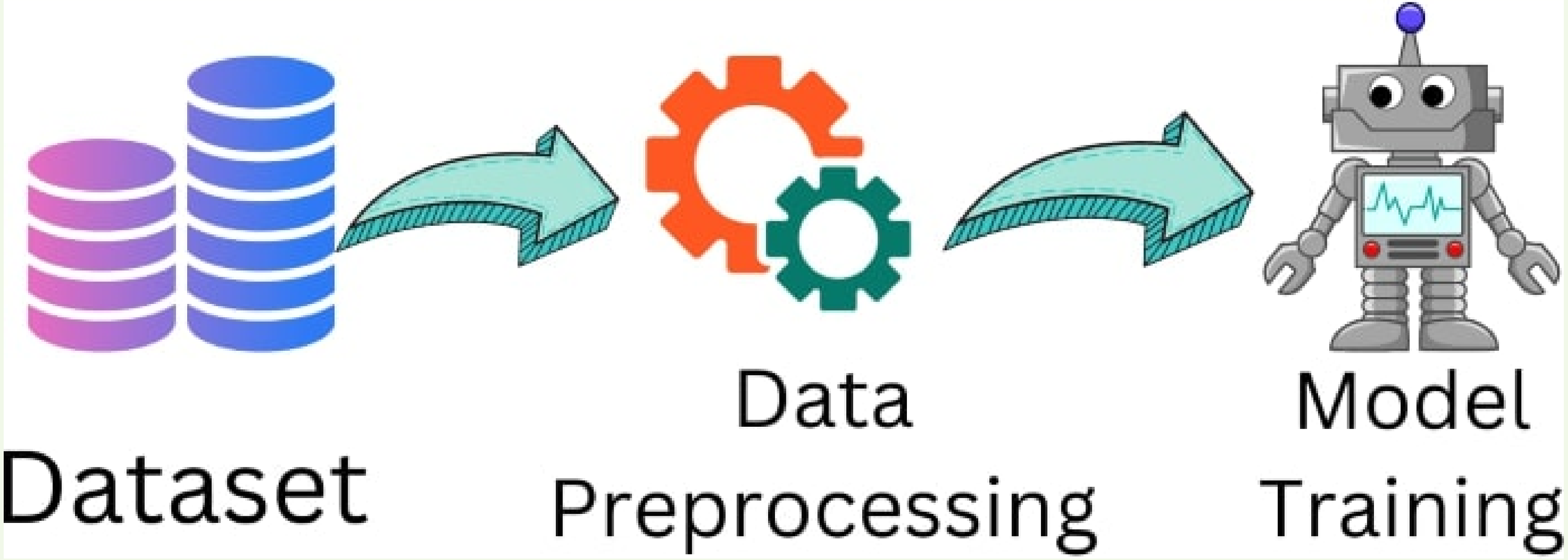
Data Preprocessing

Presented By
Sylvia Anter


Dataset
Data Preprocessing
Model Training

Data Preprocessing

Data Cleaning

Missing Values
Duplications
Outliers

Data Encoding

Convert Categorical Values to Numerical Values

Data Normalization

Standardization & Normalization

Data Cleaning

Missing Values

Find Missing Values

Handle Missing Values

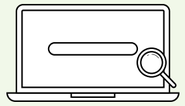
Duplications

Drop Duplicated values

Outliers

Find Outlier

Handle Outliers



Data Cleaning : Missing Values

Defualt Missing values in pandas

1- N/A

2- NAN

3- Empty Cell

4- Null

Other null values ?

df.read_csv(“ ”,na_values=[“ - “ , “ ^ “]


#10
ISNULL() & NOTNULL()
FUNCTIONS
IN PANDAS
HTML
<?PHP?>

Data Cleaning : Missing Values

isnull()	*find (NAN/None)*	True / False
isnull().sum()	*find null in each column*	Col_Name : 20
isnull().sum().sum()	*find total num of null*	40
notnull()	*find actual values*	true/false
notnull().sum()	*find actual in each col*	col_name : 30
notnull().sum().sum()	*find total num of actual*	2900
isnull().mean()*100	*% null in each col*	col_name : 25.0
notnull().mean().mean()*100	*% actual in each col*	col_name : 90.0
isnull().any()	*there is missing value ?*	col_name : true

PYTHON PANDAS

Handle Missing Data in DataFrame

itversity
making it resourceful

Data Cleaning : Missing Values

how to handle the missing values ?

1- Remove : %%

2- Replace || Filling

1- sklearn – simpleImputer()

2- pandas - fillna()

Remove :
using Dropna()

dropna()	*remove any value - null*
dropna(axis=1)	*remove any col - null*
dropna(thresh=4)	*row : at least 4 actual*
dropna(subset=[' age'])	*remove values age : missing*

Replace || Filling
using fillna()

fillna(0)	replace null : 0
df['age'].fillna(25)	*Replace M.Age 25*
df['age'].fillna(df['age'].median())	replace M.age Avg
df.fillna(method='ffill')	Replace with the forword value
df.fillna(method='bfill')	*Replace with next value*

Data Cleaning : Missing Values

Replace || Filling using sklearn

from sklearn.impute import SimpleImputer

```
imputer = SimpleImputer(strategy='mean')
df[['Age']] = imputer.fit_transform(df[['Age']])
```

```
imputer = SimpleImputer(strategy='median')
df[['Age']] = imputer.fit_transform(df[['Age']])
```

```
imputer = SimpleImputer(strategy='most_frequent')
df[['Name']] = imputer.fit_transform(df[['Name']])
```

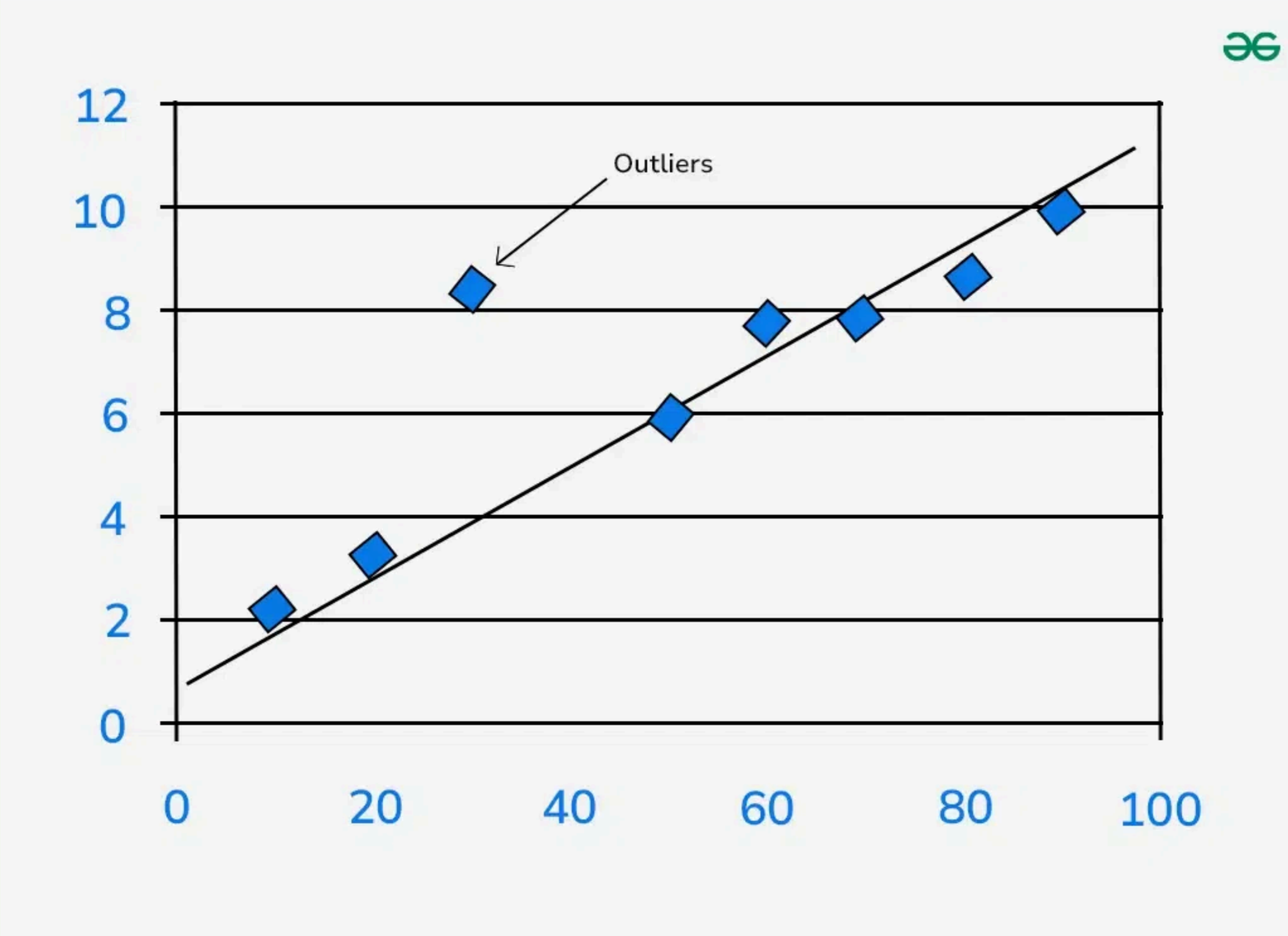
```
imputer = SimpleImputer(strategy='constant', fill_value=0)
df[['Salary']] = imputer.fit_transform(df[['Salary']])
```

Data Cleaning : Duplicated values

Duplication

Using function
drop_duplicates()

Data Cleaning : Outliers



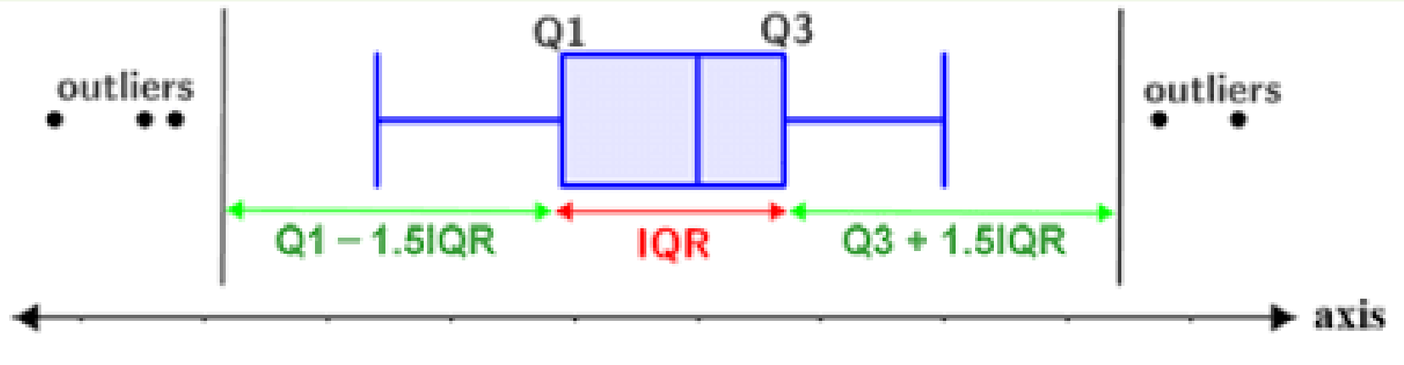
Data Cleaning : Outliers

Outliers

Using IQR

Using Log Transformation

IQR



Data Cleaning : Outliers

Q1 : 25% **Q3 : 75%**

113	110	ahmed	africa	dubi

IQR

1- GET :

Q1 , Q3

2- IQR = Q3- Q1

3- Get :

Lower_bound

upper_bound

code

```
Q1 = df[" "].quantile(0.25)
Q3=df[" "].quantile(0.75)
IQR= Q3 - Q1
Lower_bound=Q1 - 1.5*IQR
upper_bound=Q3+ 1.5*IQR
```

Range of the data

Replace each value greater than upper_bound with upper_bound .

Replace each value less than lower_boundn with lower_bound.

Data Cleaning : Outliers

Range of the data

Replace each value greater than upper_bound with upper_bound .

Replace each value less than lower_boundn with lower_bound.

code :

```
For col in df[“ “]:
    df[col] = np.where(df[col]<lower_bound , lower_bound , np.where(df[col]>upper_bound , upper_bound , df)
```

Data Cleaning : Outliers

2. Log Transformation : reduce the effect of outliers without remover them

code :

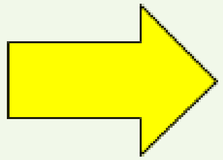
```
import numpy as np
df['salary_log'] = np.log(df['salary'])
```

note :

df.select_dtypes(include=[np.number]).columns

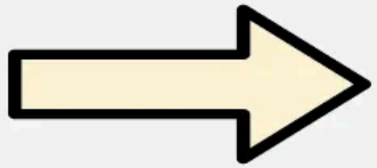
Data Encoding

Color
Red
Red
Yellow
Green
Yellow



Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

Color
Red
Green
Blue
Red



Color
0
1
2
0

Data Encoding

Categorical Data Types

Binary data

yes/ No

Male / Female

open / Close

Nominal data

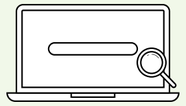
Egypt
USA
German

pandas
sklearn

Ordinal data

Low-Medium-high
s-M-L-XL-XXL

Pandas
sklearn



Data Encoding

Numeric Encoding (Label Encoder)

Used when data is binary

from sklearn.preprocessing import LabelEncoder

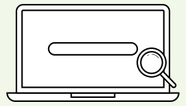
```
le = LabelEncoder()
encoded = le.fit_transform(data)
```

Numeric Encoding (OrdinalEncoder)

Used when data is ordinal

from sklearn.preprocessing import OrdinalEncoder

```
or = OrdinalEncoder(categories=
[['S','M','L']])
df[['size']] =
or.fit_transform(df[['size']])
```



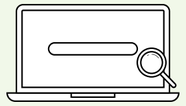
Data Encoding

Numeric Encoding (Replace)

Used when data is ordinal

pandas

```
df.replace({ 'level' : {'junior' : 0, 'senior' : 1, 'midlevel' : 2 , 'manager' : 3} },inplace=True)
```



Data Encoding

get_dummies

Used when data is Nominal

```
df=pd.get_dummies(df,dtype=int)
```

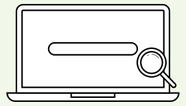
Onehotencoder

Used when data is Nominal

from sklearn.preprocessing import OneHotEncoder

```
encoder = OneHotEncoder()

df_new =
encoder.fit_transform(df[['department']])
```



Data Encoding

convert to dataframe

Used when data is Nominal

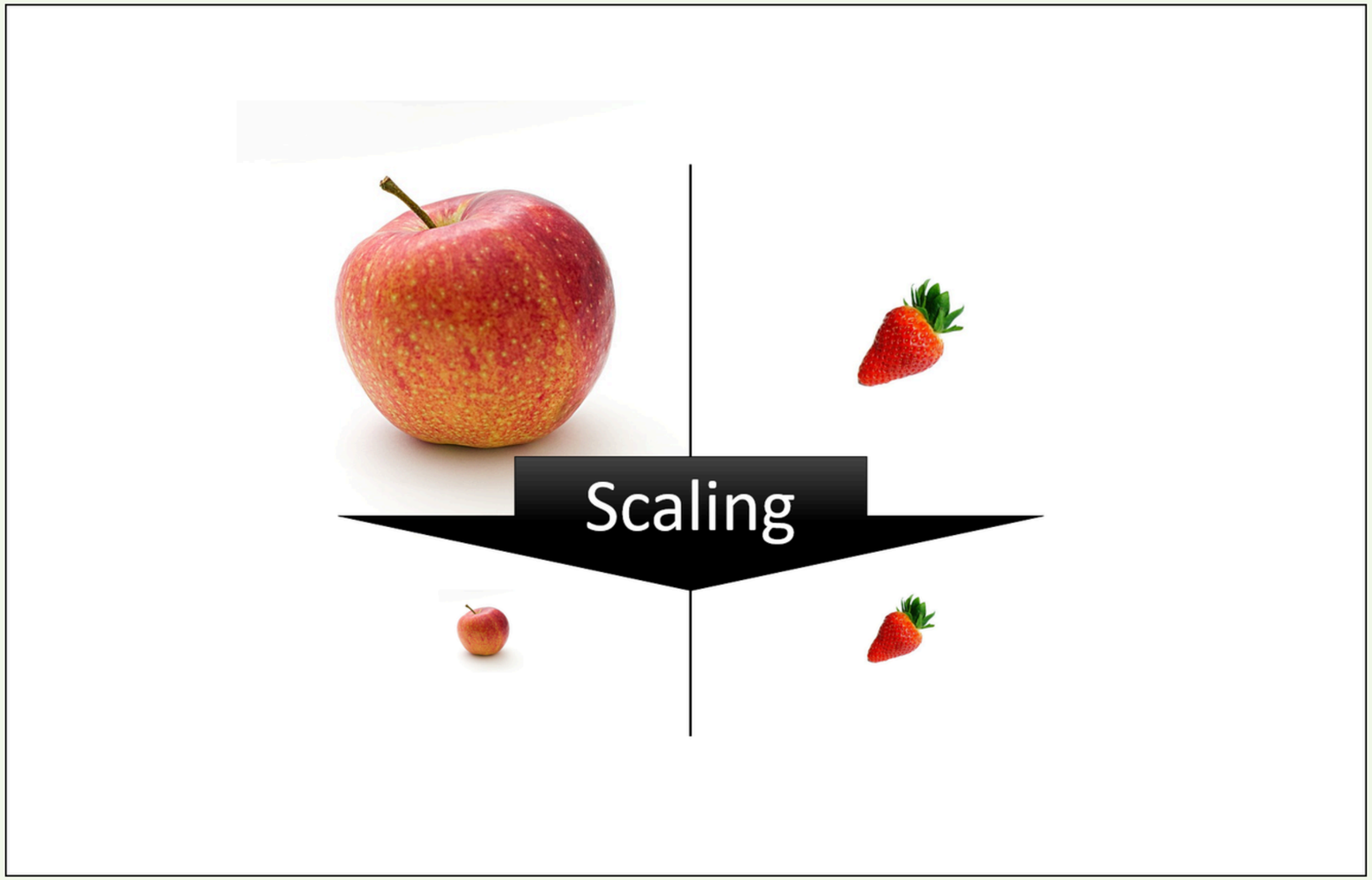
from sklearn.preprocessing import OneHotEncoder

```
encoded_df = pd.DataFrame(
df_new.toarray(),
columns=encoder.get_feature_names_out(['department']))
```

Merge df & encoded_df

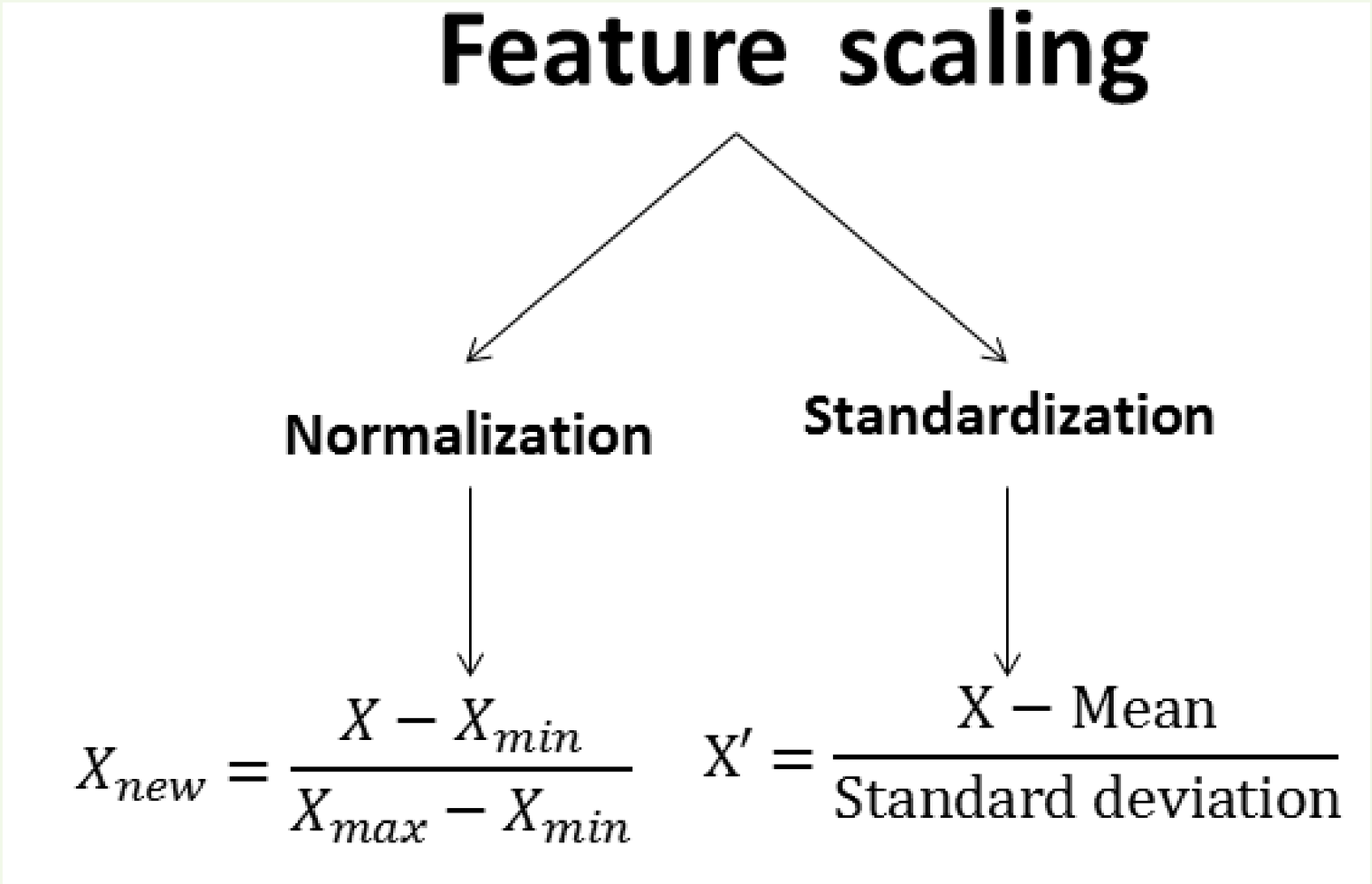
```
df_final = pd.concat([df, encoded_df], axis=1)
```

Feature Scaling



Scaling

operation that make all features (columns) at the same scale



features Scaling

Normalization

Min-Max Normalization (0 → 1 scaling)

$$X' = \frac{X - Xmin}{Xmax - Xmin}$$

Normalization is sensitive to outliers because it depends on the minimum and maximum values . You should use normalization when most of your features have a uniform distribution and don't have outliers (or if you remove the outliers first).

We should split the dataset before the step of scaling

features Scaling

Normalization

Min-Max Scaling

$$X' = \frac{X - Xmin}{Xmax - Xmin}$$

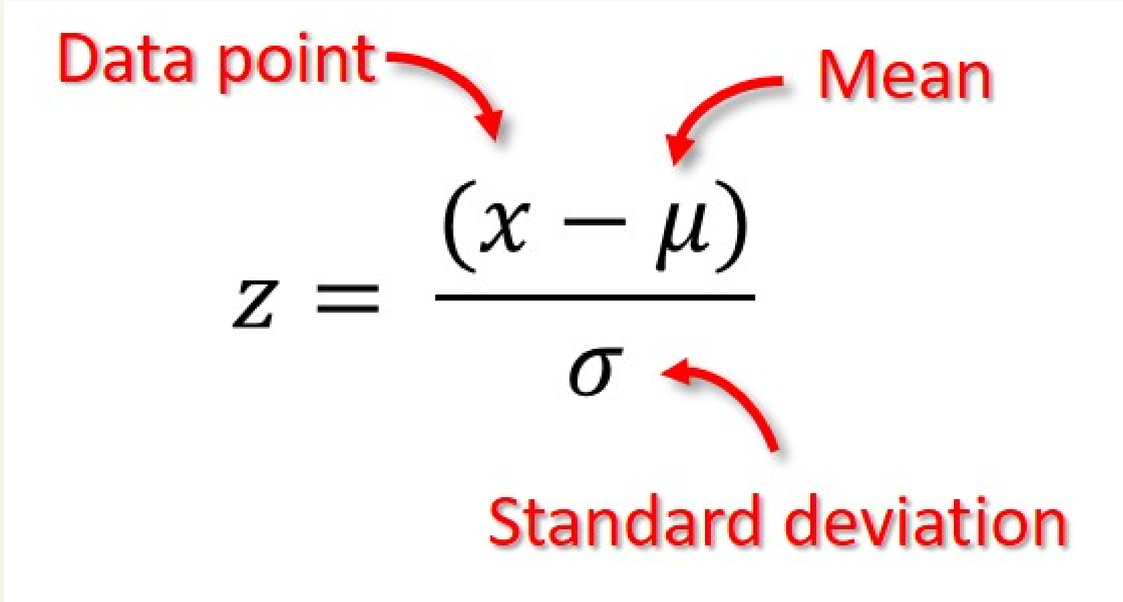
```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Min-Max Scaling
minmax = MinMaxScaler()
minmax.fit(x_train)
x_train_scaled=minmax.transform(x_train)
```

features Scaling

Standardization

(Z-score Scaling)



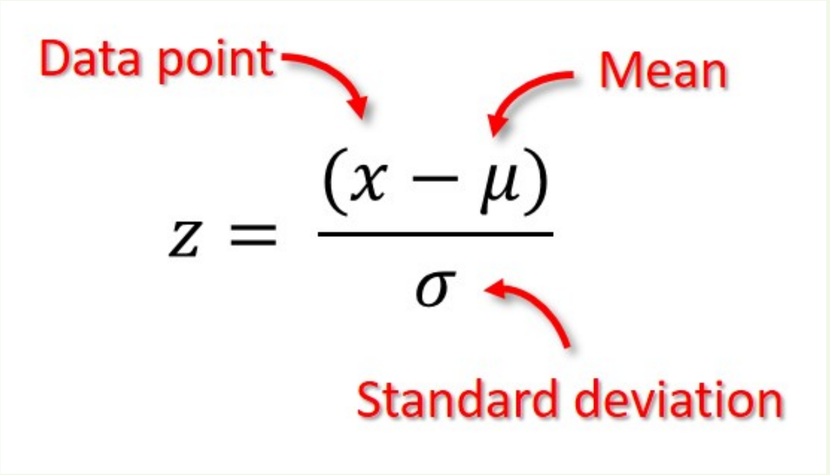
Standardization is good to outliers and can be used if your features have a normal distribution even when outliers exist.

We should split the dataset before the step of scaling

features Scaling

Normalization

Min-Max Scaling

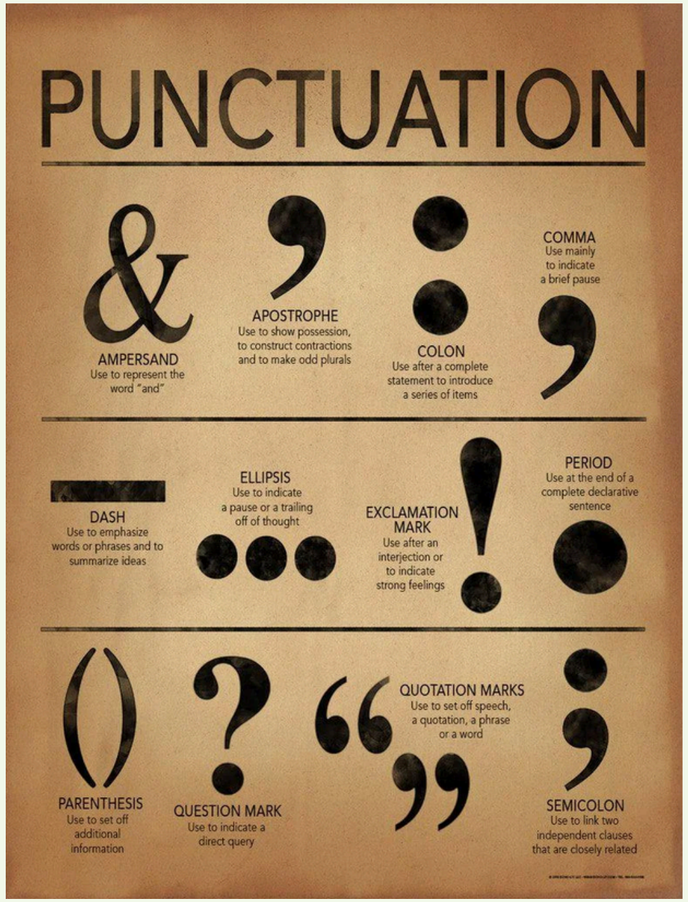



```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# z-scaling
stand = standrdscaler()
stand.fit(x_train)
x_train_scaled=stand.transform(x_train)
```

df_standard = pd.DataFrame(stand.fit_transform(df), columns=df.columns)

Remove Punctuations



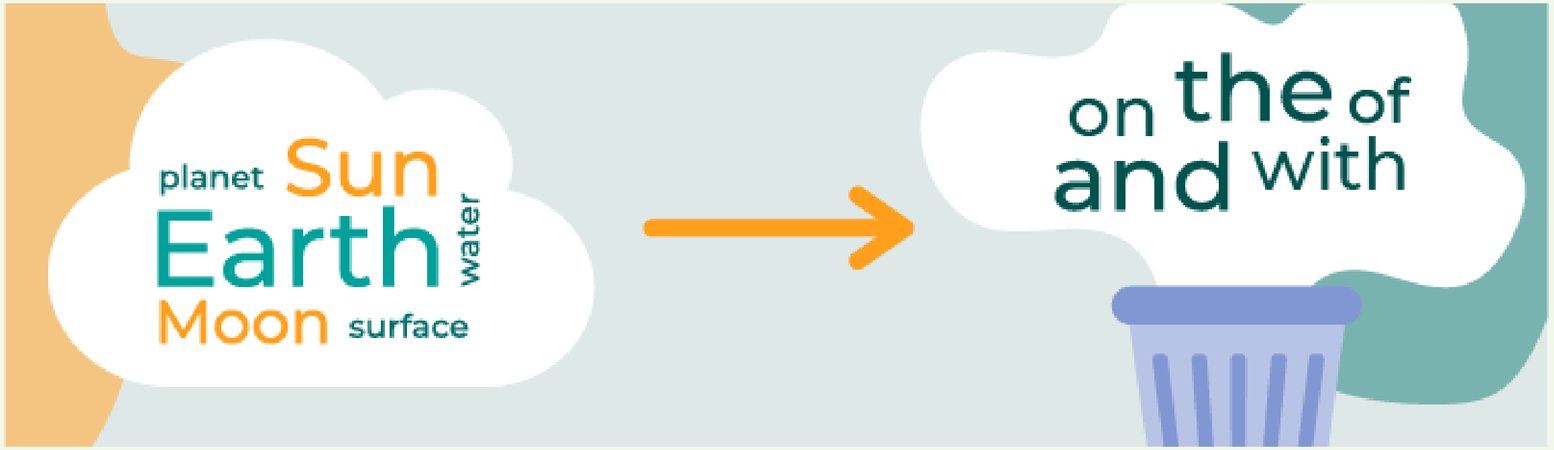
Remove Punctuations

```
def reomve_punctation(cell):
    textcell=""
    for char in cell:
        if char not in p:
            textcell=textcell+char
    return textcell
```

fruits	state	price
Apple	good!	20
apple	good	50
banana	bad	30
banana	BAD!!	60
KIWI	#bad	20
kiwi	#good	20

```
df['state']=df['state'].apply(reomve_punctation)
df
```

Remove Stop Words



Stop Words

- a
- I
- the
- in
- of
- for
- at
- to
- on
- with
- from

pizza_id	pizza_category	pizza_ingredients
1	Classic	is Pepperoni, Mushrooms, Red Onions, Red Peppers, Bacon
2	Classic	a Pepperoni, Mushrooms, the Red Onions, Red Peppers, Bacon
3	Supreme	Salami, and Tomatoes, Red Onions, Garlic
4	Supreme	Salami, Tomatoes, Red Onions,then Garlic
5	Supreme	Salami, Tomatoes, the Red Onions, then Garlic